

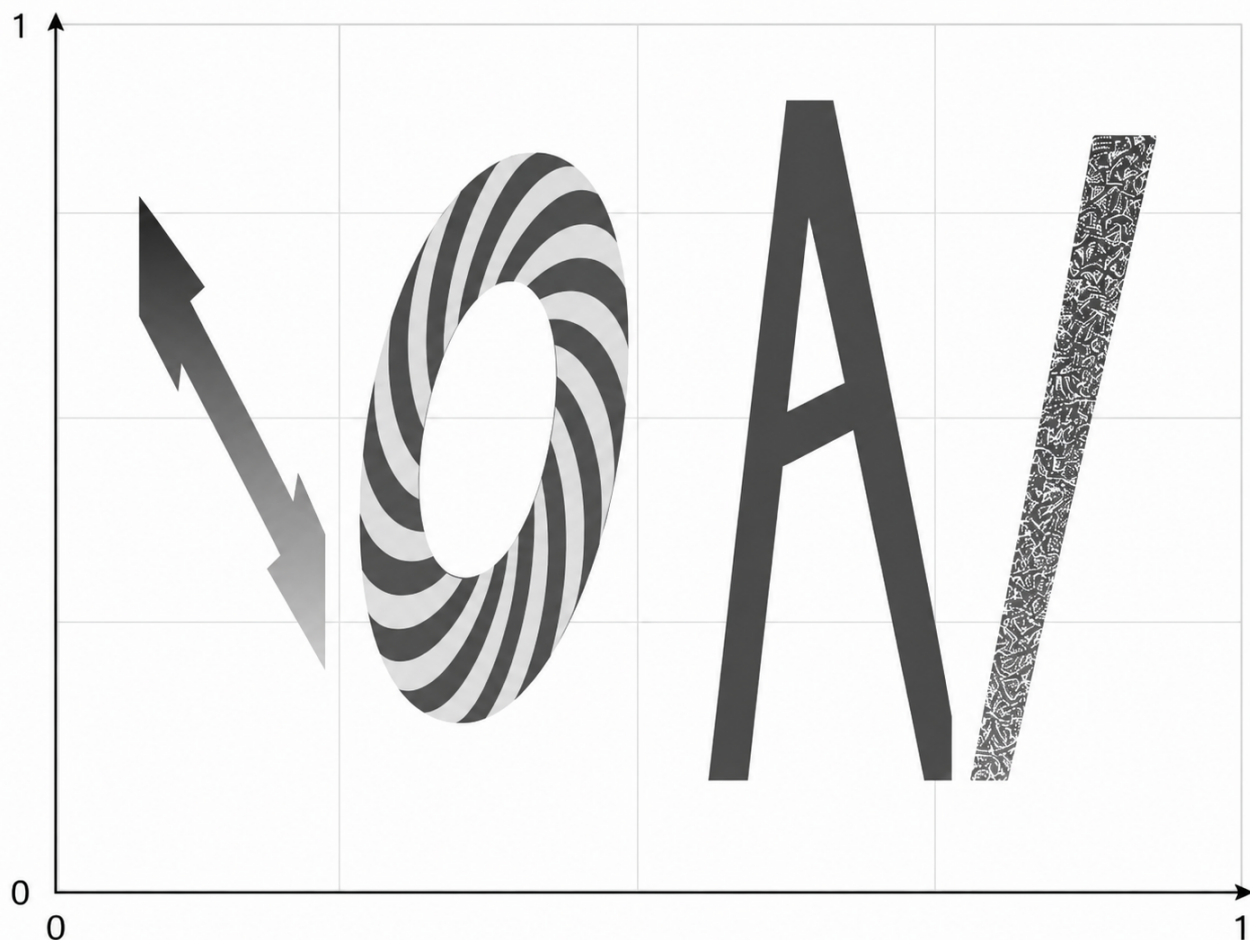
IOAI-mező

- **Időkorlát:** 5 minutes
- **Tárhely:** 5 GB
- **A megoldás mérete:** `solution.ipynb`, `custom_model.py` együtt ≤ 1 MB
- **Előre betanított modellek:** nincsenek — a nulláról kell betanítani, a kiértékelés idején nincs internet-hozzáférés
- **Alappontszám (baseline pontszám):** 31.2187

Feladat

Astana polgármestere stilizált IOAI-logókkal szeretné díszíteni a várost. Statisztikusként mindent — beleértve a logót is — $F(x, y, \overline{W})$ térbeli függvényként szemlél, ahol $x, y \in [0, 1]$ egy 2D sík koordinátáit jelölik, $\overline{W}$ pedig a stilisztikai jellemzőket, például a betűk színeit és dőlésszögeit meghatározó rejtett paraméterek halmaza.

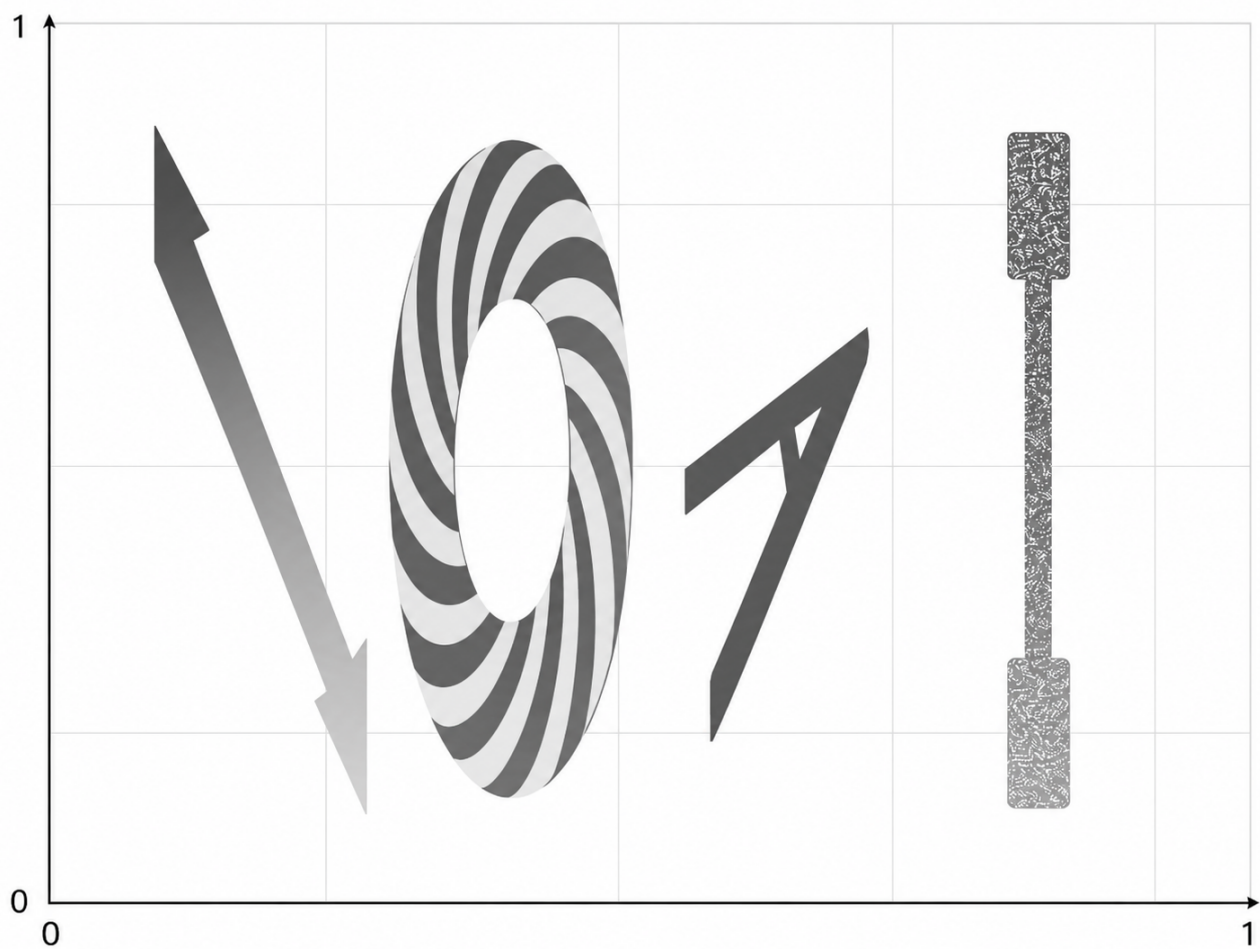
Mivel F túl összetett ahhoz, hogy explicit matematikai egyenletként fejezzük ki, az Ön feladata egy neurális hálózat betanítása annak közelítésére. A hálózat bármely (x, y) koordinátapárhoz egy értéket ad kimenetként. Így a teljes síkon a logó teljes hő térképes megjelenítését generálja. Ezt a "hő térképet" **IOAI-mező**-nek nevezzük. Az alábbi ábra példát mutat F hő térképes megjelenítésére bizonyos konkrét $\overline{W}$ (rejtett) paraméterek mellett.

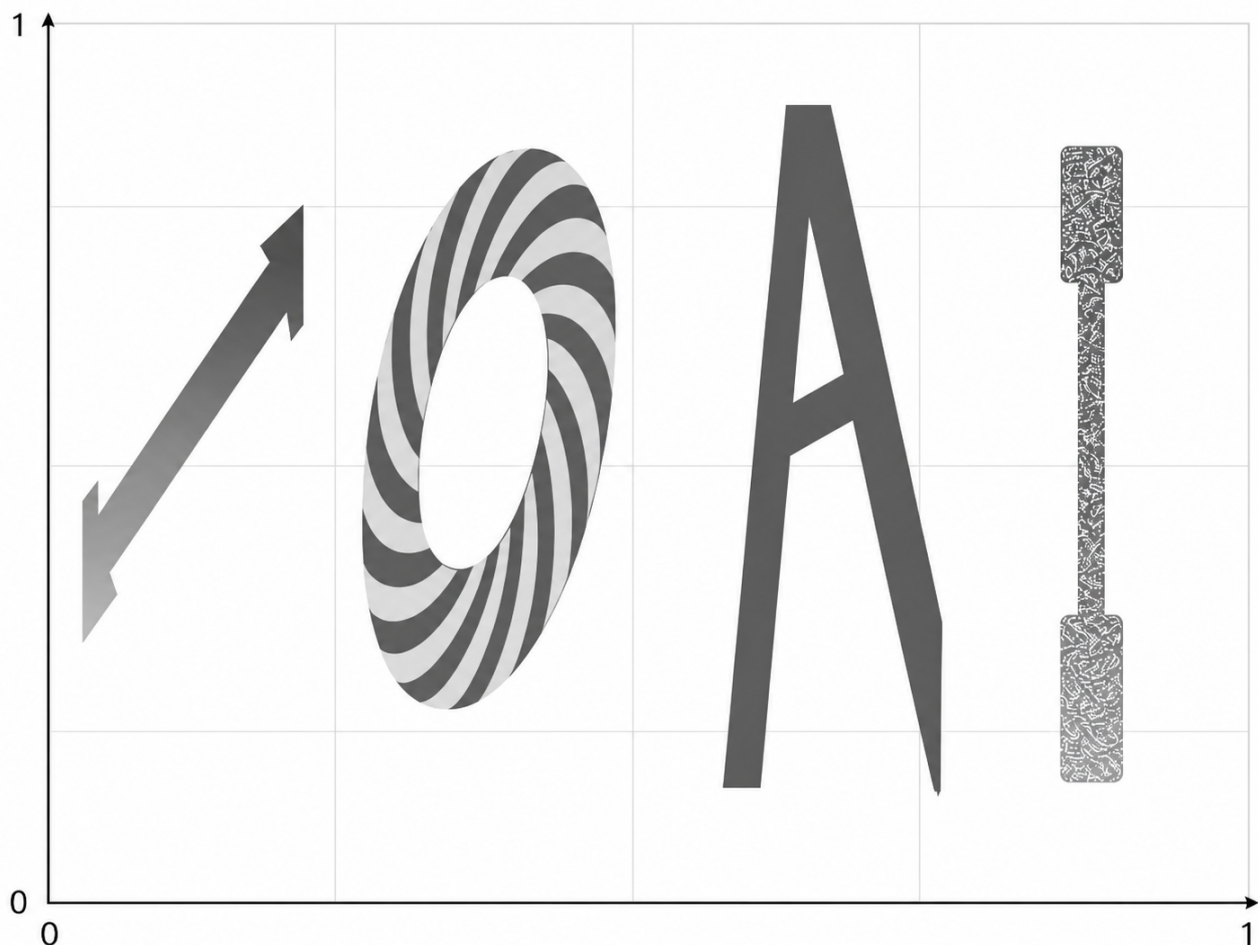


Miből áll az IOAI-mező? Öt régióból: négy betűből és a háttérből, az alábbiak szerint:

- Az első $\mathbb{I}$ betűn belüli értékek nagyon nagyok ($1e+10$, vagy nagyobbak), lineáris gradienssel
- A $\circ$ betű értékei spirális mintázatot mutatnak
- A $\mathbb{A}$ betűn belüli érték mindig -1
- Az utolsó $\mathbb{I}$ betűn belüli értékeknek a $[-2026, 2026]$ tartományból származó véletlen értékeknek kell lenniük, még akkor is, ha ugyanazon a ponton kétszer értékeljük ki őket
- A betűkön kívül az érték mindig nulla

A függvény viselkedését a $\overline{W}$ rejtett paraméterei határozzák meg. Ezek befolyásolják a betűk méretarányát és dőlését, valamint az első $\mathbb{I}$ betűn belüli értéktartományt. A betűk azonban nem metszik egymást. Az alábbiakban néhány szemléltető példa látható arra, hogyan néz ki az IOAI-mező különböző $\overline{W}$ -k mellett:





Amit megkap:

Ez a feladat NEM tartalmaz adathalmazt (datasetet). Ehelyett megkapja a generátorfüggvényt, amelyet a `data/train_config/field_config.json` helyen található JSON-konfigurációs fájl állít be.

A teszt során használt konfiguráció rejtett, de hasonló jellegű. Az Ön feladata, hogy tetszőleges mennyiségű adat felhasználásával modellt illesszen a megadott generátorra. A „tanítási” és „tesztelési” eloszlások ugyanabból a generátorból származnak — csupán azt nem tudja, hogy mely (x_i, y_i) pontokon fogjuk az eredményt kiértékelni.

A beadott megoldásnak a következőkből kell állnia:

- a betanítandó modell `custom_model.py` néven mentett osztálya. Ennek a modellnek a `torch.nn.Module` osztályból kell öröklődnie, és kizárólag `torch` importokat használhat. Egy `CustomModel` osztályt kell tartalmaznia, amelyet a `solution.ipynb` notebook használ.
- a `solution.ipynb` notebook, amely előállítja a `model.pt` súlyokat

Pontozás

Minden régió esetében a minimális pontszám 0, a maximális pontszám pedig 1. A végső pontszám az öt régióra (a négy betű mindegyikére és a háttérre) kapott pontszám átlaga, megszorozva 100-zal. Ezen felül van egy **paraméterbüntetés**:

Ha a modellnek több mint 20260 paramétere van, a pontszám feleződik.

A paraméterek számát a `sum(p.numel() for p in model.parameters())` méri. Elvárjuk, hogy a modell sztochasztikus módban is működjön, és a PyTorch `nn.Dropout` a modell része legyen.

A standard régiók esetében

Minden R régió (az első $\mathbb{I}$ betű, O , A , `Background` (háttér)) esetében a modellt $N_R = 512$ darab (x_i, y_i) tesztpontra értékeljük ki, amelyek valódi értékei v_i , az előrejelzések pedig $\hat{v}_i$. Fő metrikaként a normalizált átlagos abszolút hibát (Mean Absolute Error, MAE) használjuk. A MAE definíciója:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

A normalizálást pedig a következőképpen végezzük:

$$\text{score}(R) = 1 - \min\left(\frac{\text{MAE}(R)}{s_R}, 1\right),$$

ahol $s_R > 0$ egy skálázási konstans.

Az utolsó $\mathbb{I}$ betű régiója esetében

Ebben a régióban **a dropout engedélyezve van a kiértékelés során**. Minden j tesztpontra:

1. A modellt $K = 10$ alkalommal futtatjuk, így megkapjuk a $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ értékeket.
2. Ha bármelyik kimenet a $[-2026, 2026]$ tartományon kívül esik, akkor $\text{pointScore}(j) = 0$.
3. Ellenkező esetben kiszámítjuk a K kimenet σ_j szórását, majd pontszámmá alakítjuk:

$$\text{pointScore}(j) = \min\left(\frac{\sigma_j}{s_E}, 1\right),$$

ahol $s_E > 0$ egy rögzített skálázási konstans.

A régió pontszáma a régió összes pontjára vett átlag:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

ahol $N_E = K * N_R$.

Egyszerűen fogalmazva: minél nagyobb változatosságot ér el, annál nagyobb lesz a pontszáma ebben a régióban. Ugyanakkor a megoldás **nem használhat közvetlen véletlenszerűséget, így a PyTorch `rand*` és `_uniform` függvényeit sem. A véletlenszerűségnek az engedélyezett dropout melletti inferenciából kell származnia.**

A beadás módja

1. Nyissa meg a `solution.ipynb` fájlt, és futtassa az összes cellát!
2. Módosítsa a `CustomModel` modellt a `custom_model.py` fájlban!
3. Győződjön meg arról, hogy az utolsó cella a modellt a `model.pt` fájlba menti!
4. A JupyterLab Git lapján stage-elje, kommentelje és commitolja a `solution.ipynb` és `custom_model.py` fájlokat, majd pusholja őket!
5. Térjen vissza a Contest oldalra, és kattintson a **Submit** gombra! A beadáshoz fűzött megjegyzésnek meg kell egyeznie az előző lépésben szereplő kommenttel.